Computing Methods for Physics 1 – 11 November 2024

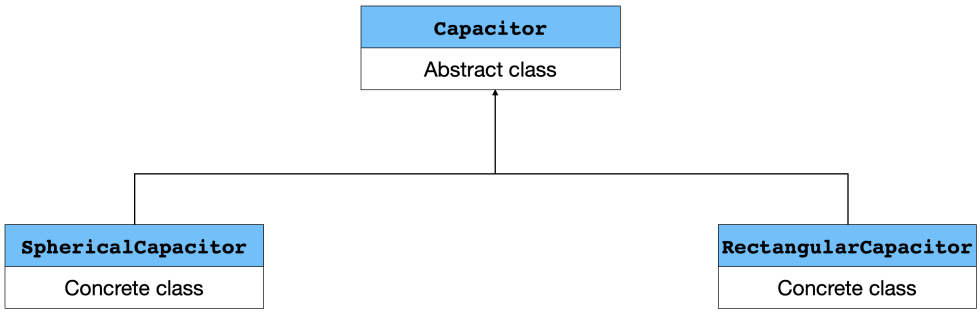
Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 16:00 as a single `zip` file.

C++ evaluation will be based on: correct syntax, proper return types, proper arguments of functions, data members and class interfaces, setters/getters, unnecessary void functions, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

Capacitors are characterized by their capacitance C, which regulates the ratio between the electric charge on their positive plate $Q(>0)$ and the differential potential $\Delta V(>0)$ between their two plates. Implement the following classes and inheritance hierarchy.



- The `Capacitor` class is an abstract class intended to represent capacitors in general; it must require concrete classes inheriting from it to provide a method to determine C, since C can be calculated from the geometry of the capacitor only once this specified, along with the properties of the material between the two plates.
- For the two concrete classes the formulas linking the geometry to the capacity are (from left to right, and ignoring higher order effects)

$$C = 4\pi\epsilon_0\epsilon_r \frac{R_1 R_2}{R_2 - R_1} \qquad C = \epsilon_0\epsilon_r \frac{S}{d}\,, \tag{1}$$

where $\epsilon_0 = 8.854 \cdot 10^{-12}\,\mathrm{F/m}$ is the vacuum permittivity, $\epsilon_r \geq 1$ is the relative permittivity of the material between the plates ($\epsilon_r = 1$ for vacuum), R_1 and R_2 are the radii of the inner and the outer spherical plates, respectively, S is the surface of the rectangular plates, and d is their separation.

- Provide the classes with constructors that take as input the geometry (e.g., R_1 and R_2), ϵ_r (assume this is a single number, i.e., the material is perfect and homogeneous), and Q or ΔV (the one *not* provided can be determined from $C = Q/\Delta V$). All capacitor instances must also have a string attribute `label`.
- Given a capacitor instance, the user must be able to print it, and find out the energy stored in the capacitor:

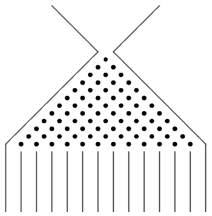
$$U = \frac{1}{2}C(\Delta V)^2 = \frac{1}{2}Q\Delta V = \frac{1}{2}\frac{Q^2}{C}\,. \tag{2}$$

- The user must also be able to know at any time the number of capacitor instances.

Part 2 – Python

Implement a class `GaltonBoard` to represent the usage of a Galton board with N rows.

The board is held vertically. Beads are dropped from the top single-bin row and bounce left or right on the pegs, with equal probability, while moving downwards from row to row. Each row has one more bin than the row immediately above it. At the end of their journeys, the beads are collected into the bins of the bottom row.



A `GaltonBoard` instance must be able to do the following.

- Represent a board and track the number of experiments undertaken on it. Think of the board as an array with $1 + 2 + \cdots + N = N(N + 1)/2$ elements (with N provided by the user) each of which is 0 when no bead has travelled.
- Simulate the evolution of a single bead from the top to the bottom: when the bead goes through a bin of the board, the array element corresponding to it increases by 1.
- Carry out the simulation of N_{exp} bead travels.
- Reset the content of all bins to 0 and lose memory of the simulations.
- Plot the distribution resulting from N_{exp} experiments.
- Print to screen the content of the bottom row at the end of the k-th simulation.
- Store the simulations to a single binary file.